

# گزارش پروژه درس اصول طراحی کامپایلر

نیمسال اول 99

موضوع پروژه :

طراحی کامپایلر برای ترجمه ی مراحل محاسبه ی یک عبارت حسابی

دانشجویان :

مرضیه رهنما (9730498)

سیده صبا ناصحی (9730495)

استاد مربوطه :

جناب آقای دکتر منصوری

به نام خدا

### پروژه درس اصول طراحی کامپایلر نیمسال اول ۹۹

طراحی کامپایلری که یک عبارت حسابی شامل اعداد صحیح (حداکثر شش رقمی)، عملگرهای +، -، \*، / و پرانتز را از صفحه کلید بگیرد و آن را به زبان انگلیسی ترجمه کرده و مراحل محاسبه عبارت را چاپ کند.

ترجمه طبق جدول زیر انجام شود:

|     |   |  |     |      |  |     |   |
|-----|---|--|-----|------|--|-----|---|
| Zer | 0 |  | Sev | 7    |  | Plu | + |
| One | 1 |  | Eig | 8    |  | Min | - |
| Two | 2 |  | Nin | 9    |  | Mul | * |
| Thr | 3 |  | Ten | 10   |  | Div | / |
| Fou | 4 |  | Hun | 100  |  |     |   |
| Fiv | 5 |  | Tou | 1000 |  |     |   |
| Six | 6 |  |     |      |  |     |   |

- مثال ها:

| ورودی کامپایلر | خروجی کامپایلر                                                                                       |
|----------------|------------------------------------------------------------------------------------------------------|
| 12925          | Assign (OneTen_Two)Tou_NinHun_TwoTen_Fiv to t1<br>Print t1                                           |
| 2840*(106+5)   | Assign OneHun_ZerTen_Six Plu Fiv to t1<br>Assign (Two)Tou_EigHun_FouTen_Zer Mul t1 to t2<br>Print t2 |

- فرض می شود که عبارت ورودی فاقد خطای کامپایلری است.

: تنها سه فاز lexical analysis، syntax analysis و intermediate code generation طراحی شوند.

: طراحی فاز lexical analysis با استفاده از ابزار Lex صورت گیرد.

: برای فاز syntax analysis و ترجمه همزمان به intermediate code از ابزار Yacc استفاده شود.

## 1. طراحی فاز Lexical Analysis با استفاده از ابزار Lex

برای اینکه بتوانیم دنباله ای از token های موردنظر را از رشته ورودی بدست بیاوریم باید ابتدا فاز Lexical Analysis را انجام می دهیم بنابراین یک فایل Lex ایجاد میکنیم و مراحل زیر را انجام میدهیم :

بخش اول فایل Lex مربوط به Regular Definition و C declaration است.

### C declaration قسمت

```
%{  
#include <string.h>  
#include "2.tab.h"  
int i = 1;  
%}
```

چون در قسمت سوم این فایل Lex (یعنی User-defined auxiliary procedures) از توابع C استفاده کردیم به همین دلیل string.h و stdio.h را include کردیم.

در فاز دوم پروژه یک فایل Yacc داریم بنابراین "y.tab.h" را هم include کردیم.

یک متغیر سراسری به نام i از نوع integer نیز تعرف کردیم که به هنگام تولید کد میانی اندیس متغیر های t را تولید میکند.

### Regular Definition قسمت

برای حالتی که یک یا بیشتر blank ( ) یا tab (t) یا new line (\n) داشته باشیم regular definition تعریف کردیم

```
delim [ \t\n]  
ws {delim}+
```

## بخش دوم فایل lex مربوط به Translation rules می باشد.

برای انواع رشته های ورودی Translation rule مینویسیم .

اگر white space داشته باشیم باید در قسمت action هیچ کاری نباید انجام بدهد از طرفی چون WS را در قسمت اول (Regular Definition) تعریف کردیم باید به صورت {ws} نوشته شود.

```
{ws} {}
```

برای اعداد 0,1,...,9 و عملگرهای +، -، \*، / و پرانتز هم Translation rule را می نویسیم .

برای مثال اگر token ، "4" را دید در attribute آن ، که از نوع string است (yyval.str) "Fou" را قرار می دهد و Four را هم return میکند به parser .

```
"4" {yyval.str=strdup("Fou"); return Four;}
```

برای سایر اعداد و کارکتر ها هم مجدد action بالا را مینویسیم.

```
"0" {yyval.str=strdup("Zer"); return Zero;}
```

```
"1" {yyval.str=strdup("One"); return One;}
```

```
"2" {yyval.str=strdup("Two"); return Two;}
```

```
"3" {yyval.str=strdup("Thr"); return Three;}
```

```
"4" {yyval.str=strdup("Fou"); return Four;}
```

```
"5" {yyval.str=strdup("Fiv"); return Five;}
```

```
"6" {yyval.str=strdup("Six"); return Six;}
```

```
"7" {yyval.str=strdup("Sev"); return Seven;}
```

```
"8" {yyval.str=strdup("Eig"); return Eight;}
```

```
"9" {yyval.str=strdup("Nin"); return Nine;}
```

```
"*" {yyval.str=strdup("Mul"); return Mul;}
```

```
"+" {yyval.str=strdup("Plu"); return Plus;}
```

```
"-" {yyval.str=strdup("Min"); return Min;}
```

```
"/" {yyval.str=strdup("Div"); return Div;}
```

```
"(" {yyval.str=strdup("("); return Lpar;}
```

```
")" {yyval.str=strdup(")"); return Rpar;}
```

### بخش سوم فایل lex مربوط به User-defined auxiliary procedures می باشد.

در این بخش دو تابع ExpCode() و NumCode() را پیاده سازی کردیم که برای تولید کد میانی به کار می روند.

این دو تابع هر جا که نیاز به چاپ کردن Assign x to y یا Assign x operand y to z داشته باشیم استفاده میشوند و بسته به اینکه این توابع چند باز صدا زده می شوند برای کد میانی t های جدید ایجاد میشود.

اگر یک عبارت حسابی دارای عملگر داشتیم از ExpCode() استفاده میکنیم آرگومان های این تابع شامل یک آرایه کارکتری برای اندیس t (char n[]) و سه تا char pointer برای به ترتیب اولین عدد (char \*num1) ، عملگر (char \*operator) و دومین عدد (char \*num2) است.

با هر بار صدا زدن تابع در قسمت parser ، i افزایش می یابد و عبارت مورد نظر با توجه به ورودی های تابع (attribute های گرامر در parser) کد میانی را چاپ می کند و در نهایت هم اندیس t را برمی گرداند.

```
char *ExpCode(char n[] , char *num1 , char *operator , char *num2)
```

```
{  
    sprintf(n , "%d" , i);  
    i++;  
    printf("Assign %s %s %s to t%s\n" , num1 , operator , num2 , n);  
    return n;  
}
```

اگر در عبارت حسابی فقط عدد داشتیم ( هیچ عملگری نداشتیم ) از NumCode() استفاده میکنیم آرگومان های این تابع شامل یک آرایه کارکتری برای اندیس t (char n[]) و یک char pointer برای عدد است .

اگر i=1 باشد یعنی فقط عدد داریم در نتیجه کد میانی را فقط برای عبارتی که عدد دارد تولید میکند و اندیس t که همیشه 1 است را return میکند اما اگر i برابر با 1 نباشد این تابع کاری انجام نمی دهد و فقط اندیس t که از قبل در آرایه کارکتری n داشتیم را return می کند.

## 2. طراحی فاز **Syntax Analysis** ترجمه همزمان به **Intermediate Code** با استفاده از ابزار **Yacc**

با توجه به **token** های بدست آمده در فاز **lexical** ، گرامر را به صورت **translation scheme** می نویسیم و با استفاده از **semantic action** ها عملیات **parsing** و ترجمه را انجام می دهیم. بخش اول فایل **lex** مربوط به **Yacc declarations** و **C declaration** است.

### قسمت C declaration

```
%{  
  
#include <stdio.h>  
  
#include <stdlib.h>  
  
#include <string.h>  
  
void yyerror (char const *s);  
  
%}
```

چون از توابع کتابخانه **C** مانند **printf ()** ، **sprintf()** و... در این فایل استفاده کردیم **stdio.h** و **stdlib.h** و **string.h** را **include** کردیم.

تابع **yyerror** در بخش سوم فایل **Yacc** (یعنی **user-defined auxiliary procedures**) قرار دارد.

### قسمت Yacc declarations

چون **yyval** به طور پیش فرض **integer** هست ، به همین دلیل یک **Union** در فایل **Yacc** تعریف کردیم (قبل از **token** ها) که در این **Union** یک متغیر از نوع **char pointer** تعریف کردیم. ( **token** ، **type** ) ها و **non terminal** ها را هم **char pointer** می کنیم )

```
%union{  
  
char *str;  
  
}Token → %token <str>Four      NonTerminal → %type <str>expr
```

Token ها (Terminal ها) و nonterminal ها ی گرامر را تعریف می کنیم.

### Token :

%token <str>Zero

%token <str>One

%token <str>Two

%token <str>Three

%token <str>Four

%token <str>Five

%token <str>Six

%token <str>Seven

%token <str>Eight

%token <str>Nine

%token <str>Mul

%token <str>Plus

%token <str>Min

%token <str>Div

%token <str>Lpar

%token <str>Rpar

### NonTerminal :

%type <str>program

%type <str>stmts

%type <str>expr

%type <str>term

%type <str>factor

%type <str>digits

%type <str>thousand

%type <str>hundred

%type <str>ten

%type <str>digit

بخش دوم فایل Yacc مربوط به Translation rules می باشد.

در این بخش گرامر را به صورت Translation Scheme نوشتیم و برای هر production ، Semantic action نوشتیم.

گرامر :

program → stmts

stmts → expr

expr → expr Plus term | expr Min term | term

term → term Mul factor | term Div factor | factor

factor → digits | Lpar expr Rpar

digits → thousand | hundred | ten | digit

thousand → digit hundred | digit digit hundred | digit digit digit hundred

hundred → digit ten

ten → digit digit

digit → Zero | One | Two | Three | Four | Five | Six | Seven | Eight | Nine

برای expr → term ، term → factor ، factor → digits و digits و digit سمت راست های production semantic action ها به صورت زیر خواهد بود زیرا فقط مقدار attribute سمت راست های production را در nonterminal ، attribute قرار میدهد.

|                   |                             |
|-------------------|-----------------------------|
| expr : term       | {sprintf(\$\$, "%s", \$1);} |
| term : factor     | {sprintf(\$\$, "%s", \$1);} |
| factor : digits   | {sprintf(\$\$, "%s", \$1);} |
| digits : thousand | {sprintf(\$\$, "%s", \$1);} |
| digit : Zero      | {sprintf(\$\$, "%s", \$1);} |



production ، ten برای اعداد دو رقمی تعریف شده بنابراین در سمت راستش 2 تا nonterminal داریم که یکی برای دهگان و دیگری برای یکان است و در قسمت Semantic action آن با توجه به ترجمه ای که صورت سوال از ما خواسته به کمک attribute های nonterminal های سمت راست رشته مورد نظر را ایجاد کردیم و در attribute ، nonterminal سمت چپ production قرار دادیم.

```
ten : digit digit {sprintf($$, "%sTen_%s", $1, $2);};
```

hundred برای اعداد سه رقمی تعریف شده و مشابه عملی که برای ten انجام دادیم را برای hundred هم تکرار کردیم .

```
hundred : digit ten {sprintf($$, "%sHun_%s", $1, $2);};
```

چون صورت سوال از ما اعداد حداکثر 6 رقمی خواسته پس برای thousand سه تا سمت راست در نظر گرفتیم

اعداد چهار رقمی

```
thousand → digit hundred
```

اعداد پنج رقمی

```
thousand → digit digit hundred
```

اعداد شش رقمی

```
thousand → digit digit digit hundred
```

در قسمت semantic action هر کدام از سمت راست های thousand ، ابتدا یک آرایه کارکتری تعریف کردیم و در خانه اول آن کارکتر '(' را قرار دادیم سپس به کمک strcat و strcpy attribute های digit ها تا قبل از hundred را در پرانتز قرار میدهم و سپس attribute ، hundred را به رشته حاصل می چسبانیم.

برای مثال این Semantic Action برای production اعداد 6 رقمی است.

```
{char string[200]; string[0] = '('; strcat(string , $$); strcpy($$ , string); strcat($$ , "Hun_"); strcat($$ , $2); strcat($$ , "Ten_"); strcat($$ , $3); strcat($$ , "Tou_"); strcat($$ , $4);}
```

Semantic action ها برای `expr` و `term` به این صورت عمل می کنند که با فراخوانی تابع `ExprCode` و دادن `attribute` های سمت راست های `production` به تابع `کد میانی` را تولید میکنند و در نهایت رشته `ti` اندیس `t` را به `attribute` ، `nonterminal` سمت چپ `production` می دهند.

به عنوان مثال :

```
expr : expr Plus term    {char n[200]; sprintf($$, "t%s", ExpCode(n , $1 , $2 , $3));}
```

```
term : term Div factor    {char n[200]; sprintf($$, "t%s", ExpCode(n , $1 , $2 , $3));}
```

اگر ورودی فقط عدد باشد در این صورت `کد میانی` در قسمت `Semantic action` ، `Production` زیر چاپ می شود و رشته `t1` در `attribute` ، `nonterminal` سمت چپ `production` قرار می گیرد.

```
stmts : expr                {char n[200]; sprintf($$, "t%s", NumCode(n , $1));}
```

و در پایان با استفاده از `production` زیر `ti print` ( `i` اندیس `t` ) چاپ میشود.

```
program : stmts              { printf("print %s", $1);}
```

**بخش سوم فایل Yacc مربوط به User-defined auxiliary procedures می باشد.**

در این قسمت تابع `yyerror` و `main` را قرار دادیم.

در تابع `main` رشته ورودی از یک فایل `text` خوانده می شود و با صدا زدن `yyparse()` عملیات `parsing` و ترجمه انجام می شود.

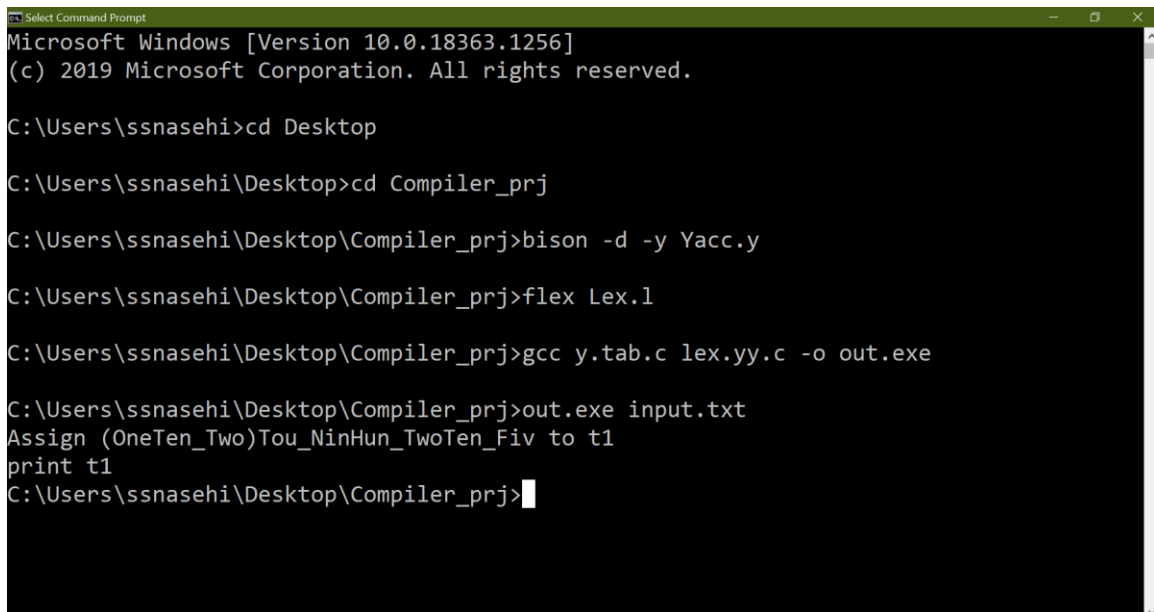
```
void yyerror (char const *s) {  
    fprintf (stderr, "%s\n", s);  
}  
  
extern FILE *yyin;  
  
int main(int argc,char **argv)  
{  
    ++argv, --argc;  
    yyin = fopen( argv[0], "r" );  
    yyparse();  
    return 0 ;  
}
```

ورودی کامپایلر :

12925

خروجی کامپایلر:

Assign (OneTen\_Two)Tou\_NinHun\_TwoTen\_Fiv to t1  
print t1



```
Select Command Prompt
Microsoft Windows [Version 10.0.18363.1256]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\ssnasehi>cd Desktop

C:\Users\ssnasehi\Desktop>cd Compiler_prj

C:\Users\ssnasehi\Desktop\Compiler_prj>bison -d -y Yacc.y

C:\Users\ssnasehi\Desktop\Compiler_prj>flex Lex.l

C:\Users\ssnasehi\Desktop\Compiler_prj>gcc y.tab.c lex.yy.c -o out.exe

C:\Users\ssnasehi\Desktop\Compiler_prj>out.exe input.txt
Assign (OneTen_Two)Tou_NinHun_TwoTen_Fiv to t1
print t1
C:\Users\ssnasehi\Desktop\Compiler_prj>
```

ورودی کامپایلر :

**$2840 \times (106 + 5)$**

خروجی کامپایلر:

**Assign OneHun\_ZerTen\_Six Plu Fiv to t1**

**Assign (Two)Tou\_EigHun\_FouTen\_Zer Mul t1 to t2**

**print t2**

**Assign (Two)Tou\_EigHun\_FouTen\_Zer Mul t1 to t2**

**print t2**

```
Microsoft Windows [Version 10.0.18363.1256]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\ssnasehi>cd Desktop

C:\Users\ssnasehi\Desktop>cd Compiler_prj

C:\Users\ssnasehi\Desktop\Compiler_prj>bison -d -y Yacc.y

C:\Users\ssnasehi\Desktop\Compiler_prj>flex Lex.l

C:\Users\ssnasehi\Desktop\Compiler_prj>gcc y.tab.c lex.yy.c -o out.exe

C:\Users\ssnasehi\Desktop\Compiler_prj>out.exe input.txt
Assign OneHun_ZerTen_Six Plu Fiv to t1
Assign (Two)Tou_EigHun_FouTen_Zer Mul t1 to t2
print t2
C:\Users\ssnasehi\Desktop\Compiler_prj>
```